LINKNET — A Structure for Computer Representation and Solution of Network Problems

By P. M. Cashin, M. R. Mayson, R. Podmore*

LINKNET is an information structure for representing any network of nodes and interconnecting arcs. The structure applies linked lists and enables list-processing techniques for problem solving with networks. The LINKNET structure has provided a concise implementation of algorithms arising in a wide variety of network problem solving, such as power system analysis, game playing programs, minimum cost path finding and the determination of certain trees and meshes in a network.

1.0 INTRODUCTION

The concept of linked-lists for the storage and manipulation of data and data structures as used in this paper is well known to computer scientists; unfortunately the use of linked-list techniques in the general run of application programs and with high-level languages such as FORTRAN and ALGOL is not so common.

This may be due, as Knuth (1968) suggests, to the fact "that many people still feel that List-processing techniques are quite complicated (so that it is necessary to use someone else's carefully written interpretive system or set of subroutines) and that List-processing must be done only in a certain fixed way". It would then seem necessary to distinguish between (a) the use of linked-lists as a means for storing and relating data in a computer memory (the subject of this paper) and (b) the use of high-level List-processing systems such as IPL-V, LISP and SLIP.

Such high-level List-processors have been designed to allow the construction and analysis of tree and list-structures in such fields as algebraic formula manipulation and computational linguistics. The symbolic structure definition and manipulation for which such systems are intended is greatly facilitated by the use of particular forms of linked-lists (e.g. binary trees, strings, rings, etc.) for the internal storage of programs and data. However, this internal storage organization is not (in general) directly available to the user and the systems are suited only to those problems whose data can conveniently be encoded into the particular forms. In addition, the arithmetic and numerical analysis capability of these systems is often limited and inefficient which further restricts their general application.

In contrast to symbolic structure manipulation problems are those where it is desired to perform numerical data analysis within a complex but relatively static structure. Many network analysis problems (e.g. path-finding, electric power system analysis, network flow problems) fall into this area where a fixed network structure is used to relate network data and to guide analysis procedures. The interest lies not in a means for efficient structure manipulation but rather in a means for representing network structure to allow efficient storage and analysis of network data.

As Knuth (1968) has noted: Although List-processing systems such as IPL-V, LISP, and SLIP are useful in a large number of situations, they impose constraints on the programmer that are often unnecessary; it is usually better to use the methods of List-processing (as described by Knuth (1968)) in one's own programs, tailoring the data formats and processing algorithms to the particular application. LINKNET follows this philosophy but has application to a whole class of problems—those involving problem solving with networks. The LINKNET scheme is not being described as a rigid protocol for network problems, but rather it is described to direct attention to the applications of List-processing to network problems by showing a framework that has proved itself very effective.

The LINKNET structure was developed and successfully applied to two problems, one of power system analysis and the other a game playing program. The common element between these two problems is that they have both to deal with networks, one in the form of bus bars and power lines, the other in the form of positions and legal moves. It quickly became apparent that the LINKNET structure was useful not only because it gave a way to represent and manipulate the networks, but also it facilitated the computational procedures used. Further applications confirmed that the LINKNET structure is well suited to a general class of problems involving such operations as searching, iterative scanning, modification and organization of any network and its attributes.

2.0 THE BASIC STRUCTURE

2.1 Graphs

A network, or graph, consists of 'nodes' which are the junction points for the 'arcs' or interconnecting lines. An example network with 4 nodes and 8 arcs is shown in figure 1. Notice that, except as may be dictated by particular applications, there are no restrictions on loops cross overs or parallel arcs, and that a "tree" with no loops is also a valid network.

**The authors carried out this work in the Electrical Engineering Department, University of Canterbury, New Zealand. P. M. Cashin is currently with C.S.I.R.O. Division of Chemical Engineering, Melbourne.* *Manuscript received October, 1970.*

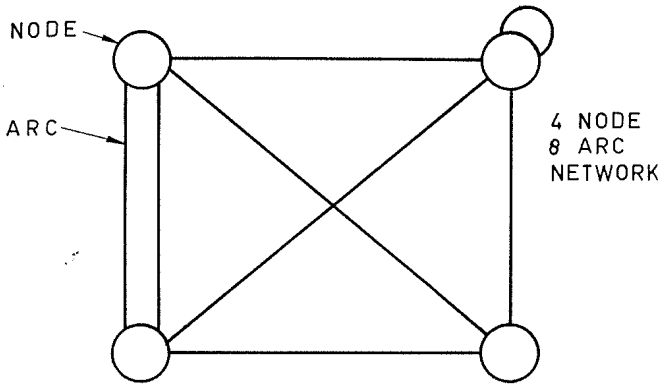


Figure 1

The nodes and arcs may each have a set of attributes; for example the arcs may have associated flow rates as in the case of a network of water pipes, or they may have directions as in the cases of a directed graph with arrows on the arcs.

2.2 LINKNET Elements

The network is represented in the computer by the LINKNET structure and although this can be implemented in machine language or in a high level language such as FORTRAN or ALGOL it is easier to develop the ideas and notation at the machine language level. Thus it will be assumed that groups of words can be assigned as contiguous blocks or 'elements', and that the addresses of these words are available to enable any of them to be accessed.

Each node of the network is represented in the LINKNET structure by a 'node-element' which holds the identity of the node and all its attributes, and in a similar manner each arc is assigned an 'arc-element'. In addition the structure has 'bead-elements' that are used to give the topology of the network by connecting the node-elements to appropriate arc-elements in a manner to be described shortly.

The elements (node-, arc- or bead-elements) can be thought of as one or more consecutive words of computer memory, with subdivision into fields, each field holding one or more attributes of the entity (node or arc) being represented. Some of the fields contain addresses rather than attributes. The address of an element, also called a link pointer, or reference to that element, is the memory location of its first word. Figure 2 shows a node-element, an arc-element and a bead-element with their fields and the names given to the fields; all the fields shown are basic to the LINKNET structure and additional fields may be specified for the purposes of particular problems.

2.3 Lists

As specified in figure 2 the LIST field of a node-element is a link to a list of bead-elements; the NEXT field of the bead-element gives a link to the next bead-element on this list, or is 'null' if there are no more bead-elements on the list. To display a list of bead-elements a diagram like that in figure 3 is used. Figure 3 also introduces a link-variable whose value is a link, in this case pointing to node 1.

Node-element:

NUMBER
LIST
NODE-DATA

Field Descriptions:

NUMBER—this field holds a number (or characters) to identify the node.
LIST—This field holds a link to a bead-element at the top of a list of beads specifying the arcs connected to this node.
NODE-DATA—This field is a composite of one or more fields holding the attributes of the node.

Arc-element:

NAME
ARC-DATA

NAME—This field holds a number (or characters) to identify the arc.
ARC-DATA—This field is a composite of one or more fields holding the attributes of the arc.

Bead-element:

NEXT
ARC
END

NEXT—This field holds a link to the next bead in a list of bead elements.
ARC—This field holds a link to an arc-element.
END—This field holds a link to the node-element of the node at the other end of the arc specified by the ARC field.

Figure 2

To refer to a field within an element the name of the field is given, followed by a link to the desired element in parentheses; for example in figure 3:

LIST(NODE)=The address of bead 1, and

NEXT (LIST(NODE))=The address of bead 2.

Notice that the fields LIST, NEXT, END, NODE-DATA and so on only have values when qualified by a link-variable (or a link-constant), they are not themselves variables.

We are now in a position to consider the LINKNET representation of a simple network, as illustrated in figure 4.

Figure 5 uses the same simple network used in figure 4 but shows the pointers (with less detail) in separate diagrams in order to bring out the way LINKNET gives a 'direct' representation of the network.

The emphasis in the construction of LINKNET is first of all on attempting to model the actual network structure and then secondly on overlaying this structure with the required network data. This 'direct' network representation enables a close correspondence between the paths taken by search and scan type algorithms within LINKNET and the graphical representation of these paths on a network diagram. This feature has proved to be a useful programming aid.

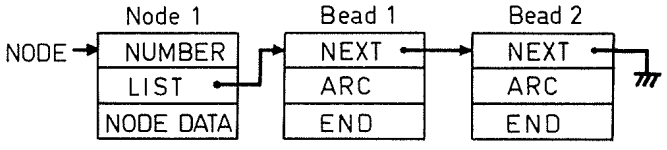


Figure 3

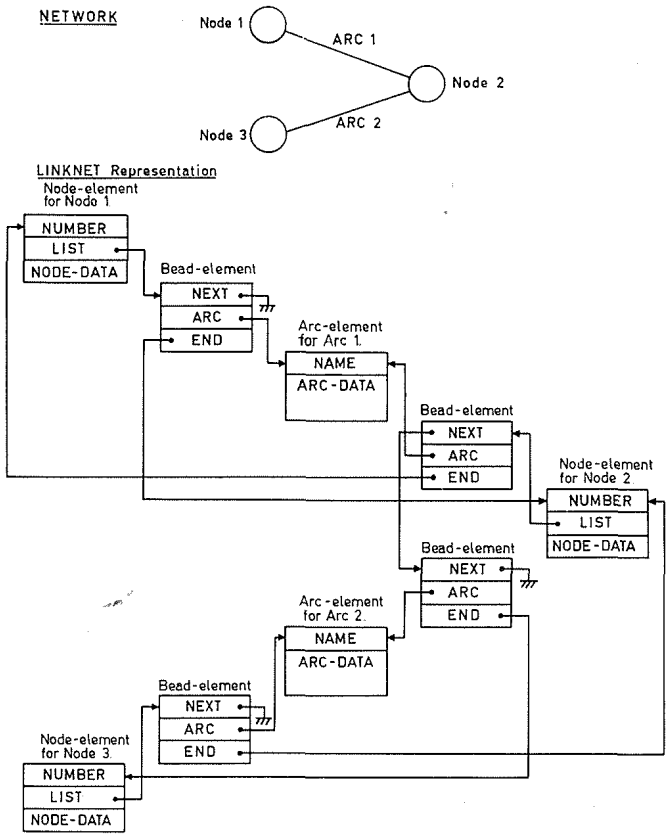


Figure 4

Two simple data access operations will now be used to illustrate the notation and show how the LINKNET structure facilitates certain procedures.

2.4 Access to Arc Attributes

The attributes of all arcs connected to any chosen node of the network can be accessed in turn by 'scanning' the list of bead-elements attached to the chosen node-element and using the link in the bead-element's ARC field. Suppose it is wished to sum the values of an attribute FLOW for each of these arcs. A pointer NODE points to (contains the address of) the chosen node-element. A temporary variable BEAD is set initially to LIST(NODE) and thus points to the first bead in the list of bead-elements connected to the node-element. ARC(BEAD) then points to the arc-element associated with the current BEAD, and FLOW(ARC(BEAD)) points to the value of FLOW for this arc-element, thus:

```
        BEAD ←LIST(NODE)
```

(A←B indicates that the variable A takes on the value of variable B)

```
        SUM ←Zero (the value of zero is 0)
START:  SUM ←SUM+FLOW(ARC(BEAD))
        BEAD ←NEXT(BEAD)
        if BEAD=null
        then exit
        else go to START
```

On exit the variable SUM contains the required quantity.

2.5 Access to Node Attributes

The attributes of all the immediate neighbours to any chosen node can be accessed in a similar manner: Example—Find the largest value of a node attribute WEIGHT for all nodes adjacent (neighbours) to a given node N.

```
        BEAD←LIST (N)
        VALUE←0
START:  IF WEIGHT(END(BEAD))>VALUE
        THEN VALUE←WEIGHT (END(BEAD))
        else continue
        BEAD←NEXT(BEAD)
        if BEAD=null, then exit
        else go to START
```

3.0 CREATION OF LINKNET

3.1 Construction of a LINKNET Structure

A simple LINKNET construction algorithm will be given in FORTRAN. The basic difference from the machine language implementation considered up to now is that the fields of each element are assigned as separate arrays or vectors, thus the fields of each element are not consecutive memory storage locations (although it may still be easier to think of them as consecutive). Also since storage for the elements is assigned as a set of arrays, identification of a node or arc can be made by specification of an index value, index i being a link or pointer to the ith node or arc element.

Let us assume that the input data that describes the network is arranged in two parts; the first giving a description of the nodes of the networks, and the second giving a description of the arcs of the network, and which nodes they connect.

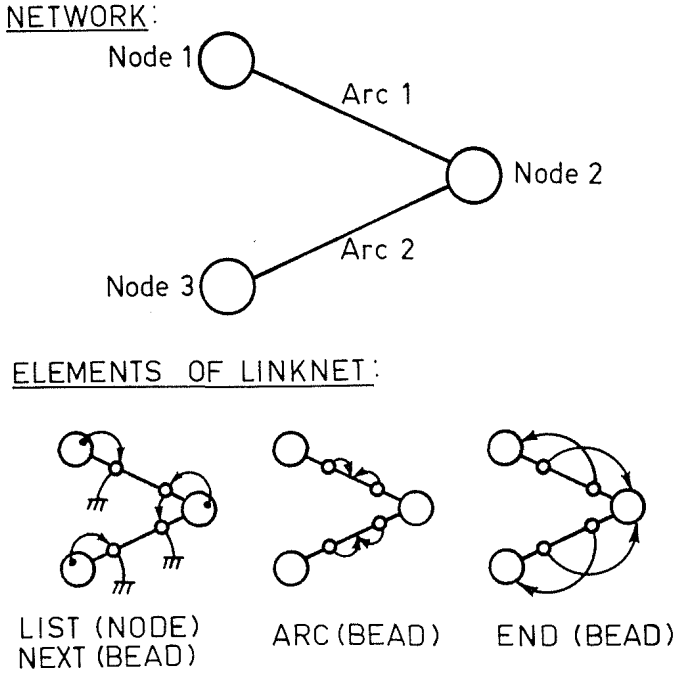


Figure 5

```
C . . . SIMPLE LINKNET CONSTRUCTION EX-
                                    AMPLE.
C . . . ALLOW UP TO 100 NODES AND 300 ARCS,
      REAL NDATA (100), ADATA (300)
      INTEGER LIST (100), ARC (600), NEXT
                          (600), END (600)
C . . . READ MAXIMUM NUMBER OF NODES
                                AND ARCS.
      READ (X, 100) NMAX, MMAX
C . . . READ NODE DATA.
      DO 1 I=1, NMAX
      READ (X, 101) N, NDATA (N)
    1 LIST (N)=0
C . . . READ ARC DATA AND NODES CON-
                                    NECTED.
      IBEAD=1
      JBEAD=IBEAD+1
      DO 2 I=1, MMAX
      READ (X, 102) M, ADATA (M), N1, N2
C  . . PUT NEW BEADS ON NODE LISTS.
      NEXT (IBEAD)=LIST (N1)
      LIST (N1)=IBEAD
      NEXT (JBEAD)=LIST (N2)
      LIST (N2)=JBEAD
C . . . SET BEAD POINTERS UP.
      ARC (IBEAD)=M
      ARC (JBEAD)=M
      END (IBEAD)=N2
      END (JBEAD)=N1
C . . . NEXT FREE BEADS.
      IBEAD=IBEAD+2
    2 JBEAD=IBEAD+1
C . . . LINKNET CONSTRUCTION COMPLETE
```

4.0 APPLICATIONS OF LINKNET

4.1 Path Finding

There are quite a number of problems that can be framed in terms of the general problem of finding a minimum cost path through a network. The arcs have associated costs and a given pair of nodes are given as start and goal (finish) for the path. Rather than considering the cost of an arc we may consider its length, the object is then to find a minimum length path from the start node to the goal node.

Particular examples of this general problem are transport routing problems, network flow problems (Berge (1962)) and control system problems (Kaufmann (1967)). An area of interest to the authors has been artificial intelligence work where network nodes may be problem states and the arcs may represent steps in a problem solution (or proof) (Pohl (1969)).

There are many algorithms that can be used to tackle the general minimum length path finding problem. One of the best known is Dynamic programming (Kaufmann (1967)) and in artificial intelligence work there is the Graph Traverser (Doran and Michie (1966)), the A* algorithm (Hart Nilsson and Raphael (1968)) and the Heuristic Path Algorithm (HPA) (Pohl (1969)). These last three are very closely related. The A* algorithm is particularly significant in that it can be shown (Hart *et al.* (1968)) that it does the minimum possible searching of the network in order to guarantee finding a minimum length path.

The algorithm used here to demonstrate LINKNET is a simplified version of the A* algorithm. The simplification is made to clarify the presentation, while still retaining the essential features of the full A* algorithm. Direct extension of the LINKNET method presented below can be used to implement the Graph Traverse algorithm, the A* algorithm or the HPA algorithm.

A Minimum Length Path Finding Algorithm

B1 Mark the start node 'open' and all the other nodes closed. Set the distance attribute of all nodes to zero.

B2 Select the 'open' node, n, with minimum 'distance'.

B3 If n is the goal node terminate the algorithm. The length of the minimum length path is the 'distance' of n, and the path can be traced back to the start by use of the predecessors or 'parents' marked at each node.

B4 Set the distance of each node m that is adjacent to node n to be the minimum of its current 'distance' value **or** the 'distance' of node n plus the length of the arc from node n to node m.

B5 Mark all the nodes that have had their distances altered by step B4 'open' and note their predecessor was n. Mark node n as 'closed' and go to step B2.

To implement the above algorithm it will be assumed that the LINKNET structure for the network has been set up and that in addition to the fields shown in figures 2 and 4 there are the following fields initialized to the values given:

DISTANCE(NODE)	=	0, this field will hold the distance value of each node as used in the B-procedure.
PARENT(NODE)	=	null, this field will hold the predecessor of the node.
STATE(NODE)	=	closed, this field will indicate if the node is 'open' or 'closed'.
OTHER(NODE)	=	null, this field holds a link to the next node in a list of 'open' nodes.
LENGTH(ARC)	=	length of the arc.

C1 (Put the start node on the 'open list'.)
OPEN-LIST←start-node (OPEN-LIST is a pointer.)

C2 (Find the minimum distance 'open' node.)
NODE←OPEN-LIST, MIN-DIST←large-value,
Loop: If MIN-DIST<DISTANCE (NODE),
Then MIN-DIST←DISTANCE(NODE),
and MIN-NODE←NODE;
In any case continue with NODE←OTHER(NODE),
If NODE ≠ null, go to loop.

C3 (Check if algorithm terminates.)
If MIN-NODE=goal-node, terminate algorithm.

C4 (Update distance of nodes.)
NODE←MIN-NODE, BEAD←LIST(NODE),
Begin: NEW-DIST←DISTANCE(NODE)+
LENGTH(ARC(BEAD)),
NEW-NODE←END(BEAD),
If DISTANCE(NEW-NODE)>NEW-DIST,
Then DISTANCE(NEW-NODE)←NEW-DIST,
and PARENT(NEW-NODE)←NODE,

and if STATE(NEW-NODE)=closed,
then STATE(NEW-NODE)←open,
and OTHER(NEW-NODE)←OPEN-LIST,
and OPEN-LIST←NEW-NODE;
In any case continue with
BEAD←NEXT(BEAD),
If BEAD ≠ null, go to Begin.

C5 (Delete node from 'open' list.)
If OPEN-LIST=NODE, then
OPEN-LIST←OTHER(NODE),
Else N←OPEN-LIST, (N is a link variable)
Step: If OTHER(N)=NODE,
then OTHER (N)←OTHER(NODE);
Else N←OTHER(N), go to Step.
After this go to step C2.

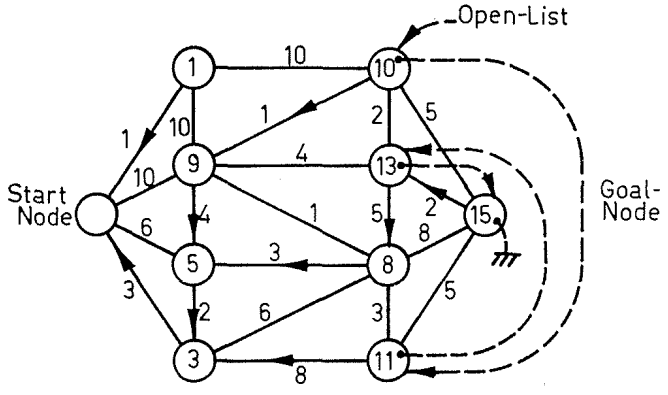
Figure 6 shows an example minimum length path finding problem with an indication of the distance values and 'open' list members at the stage when the algorithm has been through step C5 six times and has three times more to execute step C5 before terminating. The arrows on the arc indicate the parent nodes for each node, on termination a path could be traced back from the goal node to the start node using these pointers. The minimum length path, and distance of the goal node on termination of the algorithm, is 14 units.

This implementation of the minimum length path algorithm shows that the programming can very closely follow the algorithm's descriptive terms. This is a common feature in algorithms that apply to networks since very frequently they 'look at' adjacent nodes or expand out from a node; with the LINKNET structure this is simply a matter of scanning down the bead-element list attached to the node being considered.

The example used in figure 6 appears in Berge (1962), along with some different algorithms for minimum length path finding.

4.2 Formation of the Mesh Equation of an Electric Network

The analysis of a power system network, whether it be for the purpose of studying load flow, short circuits, or transient stability, requires that the network behaviour should be described by a set of simultaneous independent equations. This description can either be in the form of the network nodal current equations or in the form of the network mesh voltage equations. The network mesh voltage equations have the form: V=ZI, where V is a vector of mesh voltages, Z is the mesh impedance matrix, and I is a vector of mesh currents. Whereas the network nodal equations are easily established a considerable amount of computation has, in the past, been required to establish the network mesh equations, in particular the mesh impedance matrix. In this section we show how the LINKNET structure allows a new more efficient method for establishing network mesh equations and consequently places this form of network description in a more favourable position for the analysis of power system networks.



The numbers on the arcs are the arc lengths.
The numbers at the nodes are the distances.
The arrows on the arcs are the parent pointers.

Figure 6

The LINKNET Procedure

The LINKNET structure represents the network information in such a way that the mesh equations can be automatically set up by the direct inspection of the network meshes. The method is similar in principle to the manual method of mesh inspection which is commonly applied by engineers to directly establish the mesh equations of small uncomplicated networks.

The procedure of the method can be divided into four steps:

(i) The set-up of the network information as a LINKNET structure within the computer.

(ii) The specification of the network meshes within the LINKNET structure.

(iii) Formation of the elements of the mesh impedance matrix by inspection of the network meshes.

(iv) Formation of the elements of the vector of mesh voltages by inspection of the meshes.

The network information is read in one branch at a time and is stored within the LINKNET structure in the manner which was described in section 3.1.

The selection of the network meshes is facilitated by defining a network tree which is a subset of the network branches just sufficient in number to connect all the network nodes (Happ (1966)). The branches in the tree are termed tree branches and the remaining branches are termed link branches or links. Each link branch in the network can be associated with a closed mesh. This closed mesh consists of the link branch itself and the single path of tree branches between the ends of the link branch.

To set up a tree within the LINKNET representation of the network the algorithm 'expands' out from a root node and labels the branches as 'tree branches' and the nodes at the ends of these branches as 'tree nodes'. Subsequent expansions occur from nodes that are members of the tree, and if any branch leads to a node already in the tree then this branch is labelled as a 'link branch' rather than a 'tree branch'. The algorithm terminates after all nodes have been expanded once. A flow diagram of the program to set up the tree within the LINKNET structure is shown in figure 7. As each new node is added to the tree a special pointer TREE(NODE) is set up to indicate the tree branch which lead to this tree node. It is in

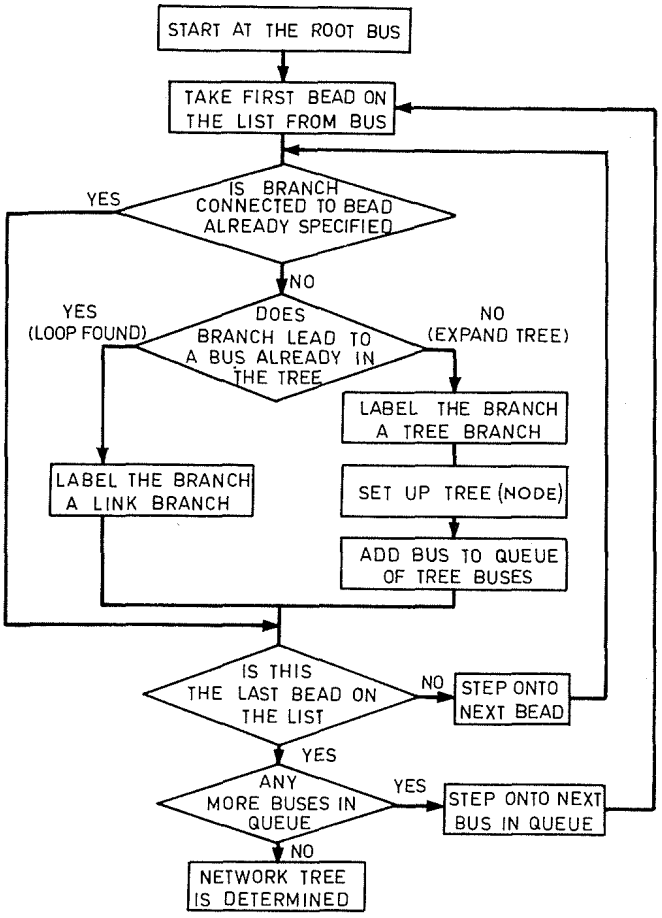


Figure 7: Flow diagram of program to set up the network tree.

fact convenient to have TREE(NODE) indicating the bead connecting the new tree node and the tree branch rather than the branch itself.

Figure 8 shows the resulting network tree and TREE(NODE) pointers when the algorithm is applied to a simple network. It should now be clear from this figure that the TREE(NODE) pointers allow the mesh associated with any link branch to be traced by starting with the nodes at each end of the link branch in turn and performing a series of:

BEAD←TREE(NODE)
BRANCH←ARC(BEAD)
NODE←END(BEAD)

operations until the root node or a node common to the two paths is reached.

Now that the procedure for tracing the network meshes has been described we are in a position to describe how the mesh impedances and mesh voltages can readily be formed by the inspection of the meshes.

We shall consider two meshes p and q, and describe the procedure for finding the mutual impedance between these meshes, Z_{pq}.

(i) Trace through mesh p and label all the branches in this mesh.

(ii) Trace through mesh q and search for labelled branches. For each labelled branch encountered: add the branch impedance to the mesh impedance if the directions of mesh p and mesh q through the branch coincide; subtract the branch impedance from the mesh impedance if the directions of mesh p and mesh q through the branch are opposite. The final value of the mesh impedance is the mutual impedance between meshes p and q. The voltage of mesh p is simply formed by tracing through mesh p and algebraically summing the branch voltage sources according to their direction with respect to mesh p. Note that to enable a concise and uncomplicated description we have not considered the possibility of inductive mutual coupling between branches. This can be handled by the LINKNET structure and the procedures outlined above.

5.0 ADVANTAGES OF LINKNET

As a basis for comparison we will compare LINKNET with a matrix array approach. In a matrix approach an N x N array is used to hold the arc data attributes. Let element ARC-DATA(I,J) be the value of the attribute for the arc joining nodes I and J. Further it will be assumed that some distinct value, say 0, indicates the absence of an arc, thus avoiding any explicit connection matrix.

While a simple matrix method does not represent the best competition to LINKNET it is a useful basis for comparison. In the case of forming power system mesh equations, as discussed in section 4.2, the methods presented in the literature (Elderman (1964)), Stagg and El-Abiad (1968) are directly based on implementation of a matrix transformation originally introduced by Kron (1939).

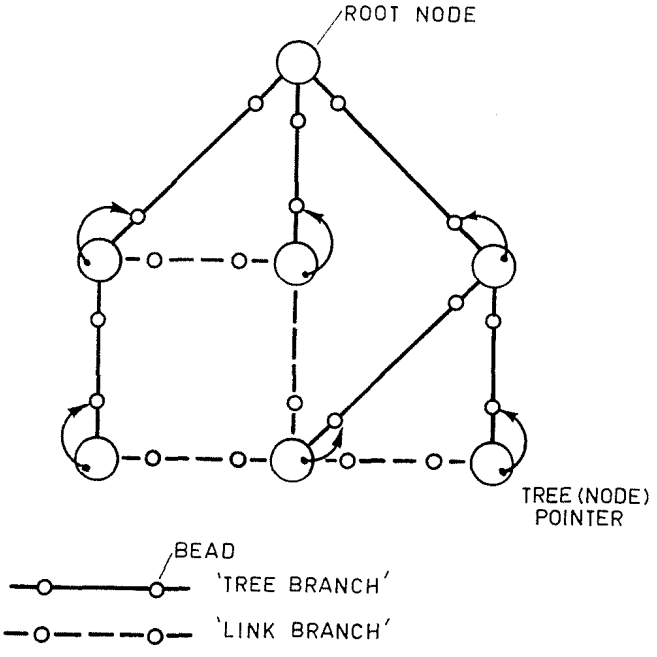


Figure 8: Representation of network tree within the Linknet structure.

5.1 Basic Storage Requirements

Vector or Array	Matrix Method	LINKNET Method
NODE-DATA	N	N
LIST	—	N
ARC-DATA	N^2	M
NEXT	—	2M
ARC	—	2M
END	—	2M
Total Storage	$N^2 + N$	$7M + 2N$

Let R=the average number of arcs per node
=M/N
Comparative storage requirements:
Matrix method=N cells
LINKNET method=7R+1 cells.
For a power supply network typical values are:
N=100 buses, R=3 lines/bus. This gives a comparative storage requirement of—
Matrix method=100 cells
LINKNET method=22 cells.

For networks with more than a single arc-attribute the comparative storage will be even more favorable to LINKNET. Also it has been assumed that one cell of memory is used for each field. This will not normally be true since the node and arc data may take 4 or 8 bytes (8 bits) as real or complex values, while the bead fields need only have values in the range 0 to M+1, say 1 or 2 bytes. Again this gives an additional advantage to LINKNET.

Another point is that a simple matrix technique can not directly represent parallel arcs between same pair of nodes, whereas LINKNET has no such restriction.

5.2 Process Speed

As an indication of process speed a typical network operation will be considered. The operation chosen is the acess of all nodes that are adjacent (neighbours) to some specified node. In the minimum length path finding algorithm discussed in section 4.1 this operation was the heart of the procedure (step B4).

For a matrix storage scheme the access operations may appear in FORTRAN as:

```
      DO 2 J=1, N
      IF (ARC-DATA (I,J).EQ.0) GO TO 2
      .............
    2 CONTINUE
```

where—I is the node being considered, and
J ranges over all its neighbour nodes.

In a LINKNET program the corresponding code is:

```
      IBEAD=LIST(I)
    1 IF(IBEAD.EQ.0) GO TO 2
      J=END(IBEAD)
      .........
      IBEAD=NEXT (IBEAD)
                —
      GO TO 1
    2 CONTINUE
```

The comparative access operations involved are:
Matrix method=N accesses to an N x N array.
LINKNET method=2R accesses to an N x 1 vector.
where R = M/N
M = the total number of arcs
N = the total number of nodes.

For a typical power system problem with N=100 and R=3, the comparative operations would be:
Matrix method=100 double index accesses
LINKNET method=6 single index accesses.

An idea of the speed of a LINKNET type program can be given for the power system problem discussed in section 4.2. The IEEE 118 bus test system was used with shunt elements neglected, leaving a network with 118 nodes and 179 arcs. The execution time to form the 62 mesh equations on an IBM 360/44 was approximately 3 seconds.

6.0 CONCLUSIONS

The LINKNET structure for representing networks by the use of linked lists has been described in some detail. The structure represents the network in the computer without any immediate regard to processing that may occur on the network or on attributes of network elements. The structure is put forward as a basic form that can be elaborated as required to include features that are special to any particular network.

After the LINKNET structure is established the implementation of several procedures is considered. It is shown that for particular problem procedures the LINKNET structure can be easily extended to facilitate the desired manipulations of the network or its attributes. More than this it is often the case that the LINKNET structure can be used to guide the course of the procedures from node to node and arc to arc in the required manner.

The use of linked-lists to represent network structure and thus relate network data has two principle advantages. Firstly, the inter-relationships between data are explicitly given in the form of 'links' rather than implied by position or ordering in storage (c.f. conventional matrix or array storage). Thus storage organization is freed from the necessity to maintain symmetry (so that relative position of data is maintained) or to preserve any particular orderings. The 'linking' of data also allows different sub-lists within the total structure to share common data or blocks of data thus aiding efficient use of storage area.

Secondly, a great many network analysis procedures and algorithms involve step-by-step operations such as searching, path-tracing, and numerical iteration, all of which are dependant on and are guided by the network structure i.e. at any stage in analysis the next operation is governed by the current point in the network structure and by the connections between this point and other points in the network. The 'links' between network data are ideally suited to providing this guiding structure.

In order to take advantage of these properties of linked-list storage but without the constraints that would be imposed by high-level list-processing systems a linked-list structure LINKNET has been developed using the basic ideas embodied in linked-list storage.

The main aim of this paper has been to bring attention to the ease of applying linked-lists and List-processing methods to network problems. It is hoped that the LINKNET structure and network procedures demonstrated here will give more programmers an